

# **Drag Your GAN : Interactive Point-based Manipulation on the Generative Image Manifold**

SIGGRAPH, 2023

이미지 처리팀

류채은 안종식 조경진

# CONTENTS

**01** | Introduction

**02** | Prerequisite

**03** | Methods

**04** | Experiments

**05** | Conclusion

**01**

# **Introduction**

# 01. Introduction

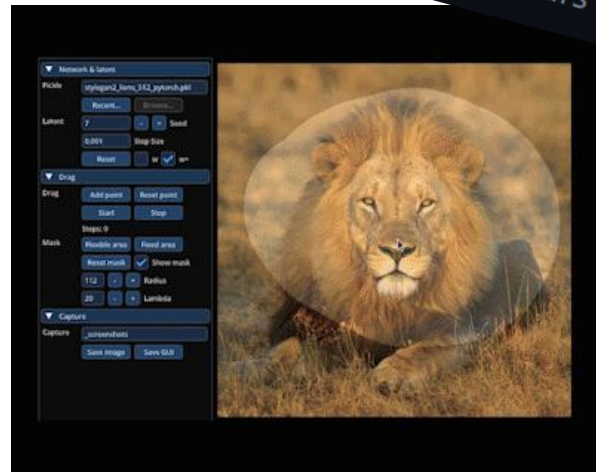
## 1. Generative AI



Diffusion Network



Kaiber



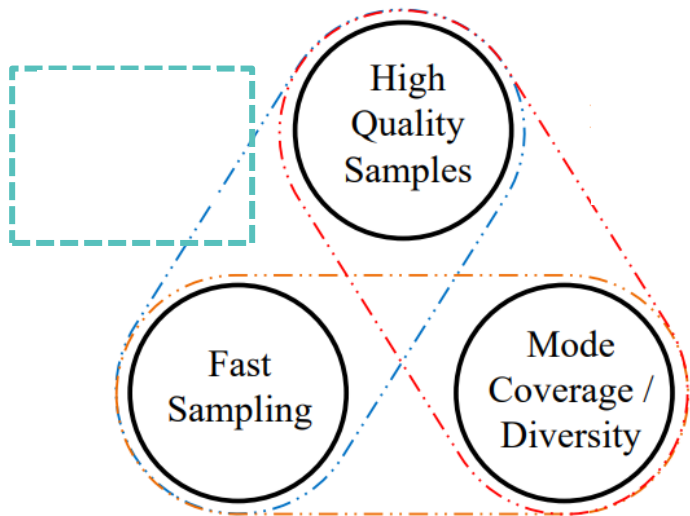
DragGAN

☆ 30.7k stars

# 01. Introduction

## 1. Generative AI

### Generative learning trilemma



- ▶ **High Quality** : 생성된 이미지의 품질
- ▶ **Fast Sampling** : 이미지 생성 시간
- ▶ **Mode Coverage** : 생성된 영상의 다양성



조경진-이미지처리 오후 2:55

하늘아래 새로운 게 없다고 바로 DragDiffusion 논문이 나왔네요 <https://arxiv.org/pdf/2306.14435v1.pdf>



2

# 01. Introduction

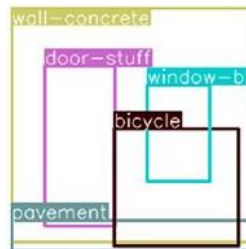
## 1. Generative AI

### ► Text to Image

An extremely angry bird



### ► Mask to Image

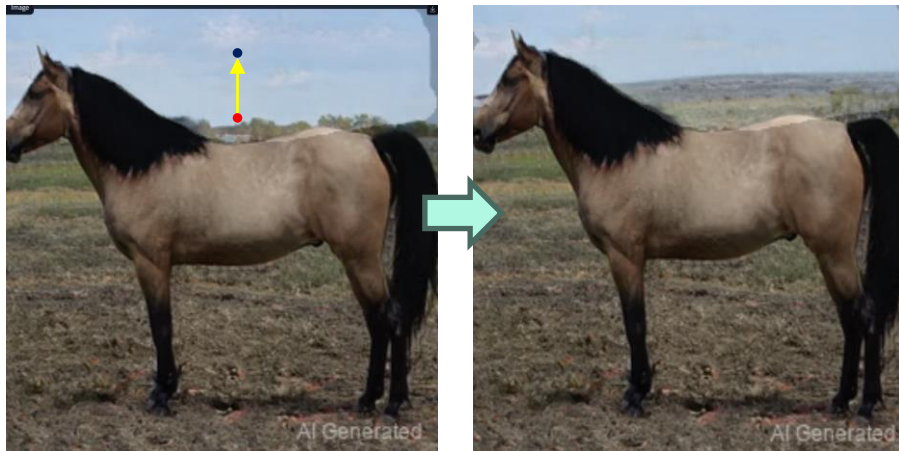


# 01. Introduction

## 1. Generative AI

### ▶ Point to Image (Drag)

- 1) Flexibility : “Pose, Shape, Expression, Layout ..”, 여러가지 형태를 변화 가능
- 2) Precision
- 3) Generality : 하나의 Class뿐만 아니라 다양한 Class에 대응 가능



**02**

# **Prerequisite**

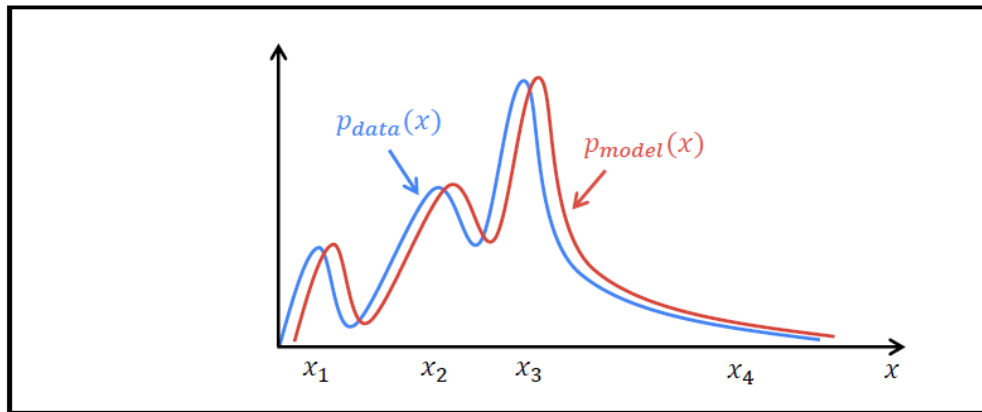


# 02. Prerequisite

## 1. GAN (Generative Adversarial Network)

The goal of the generative model is to find a  $p_{model}(x)$  that approximates  $p_{data}(x)$  well.

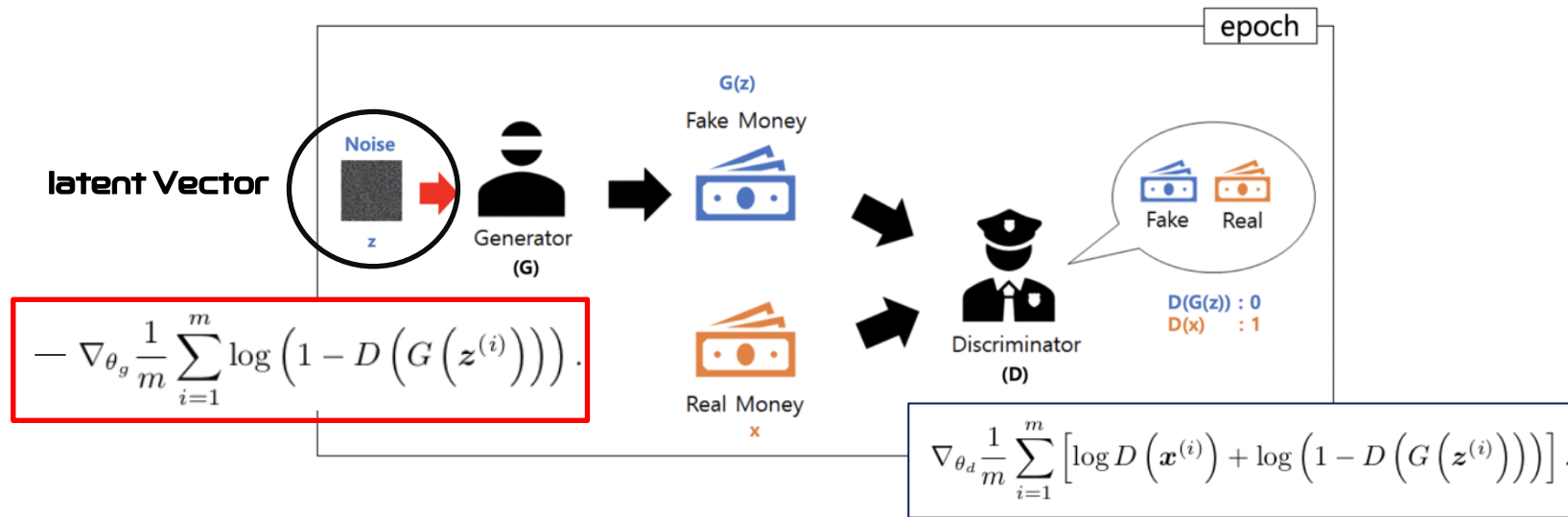
↪ *Distribution of actual images*



▶ GAN의 목적은  $P_{model}$ 과  $P_{data}$ 이 같은 Distribution으로 만드는것

# 02. Prerequisite

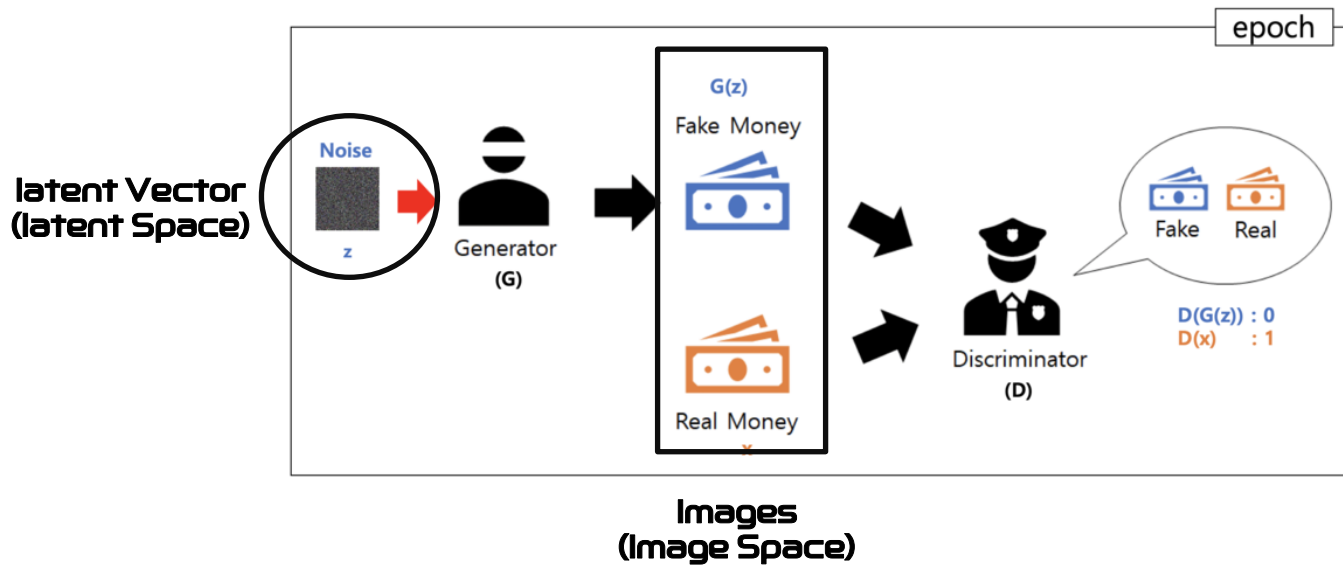
## 1. GAN (Generative Adversarial Network)



$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

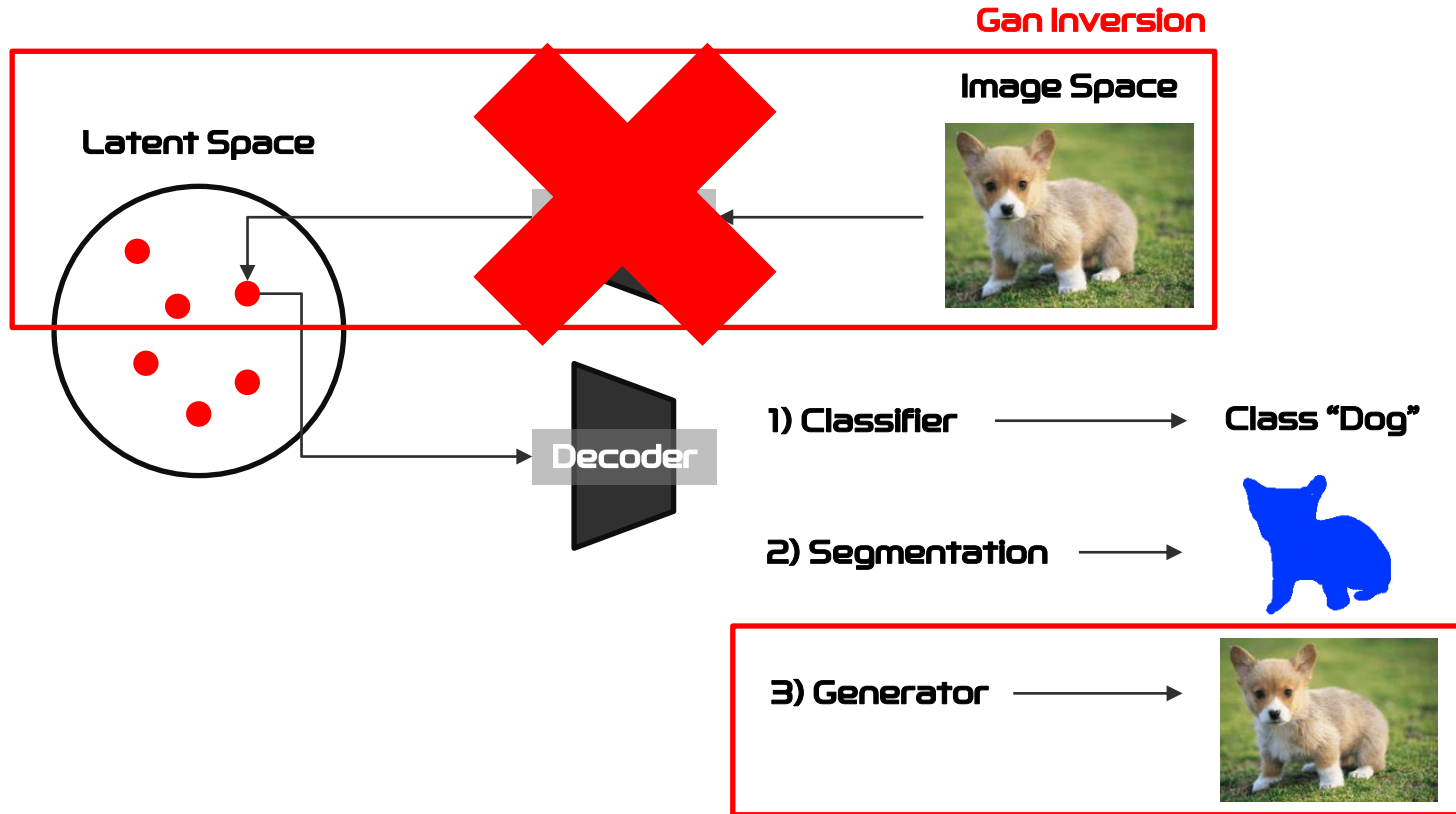
# 02. Prerequisite

## 1. GAN (Generative Adversarial Network)



# 02. Prerequisite

## 1. GAN (Generative Adversarial Network)



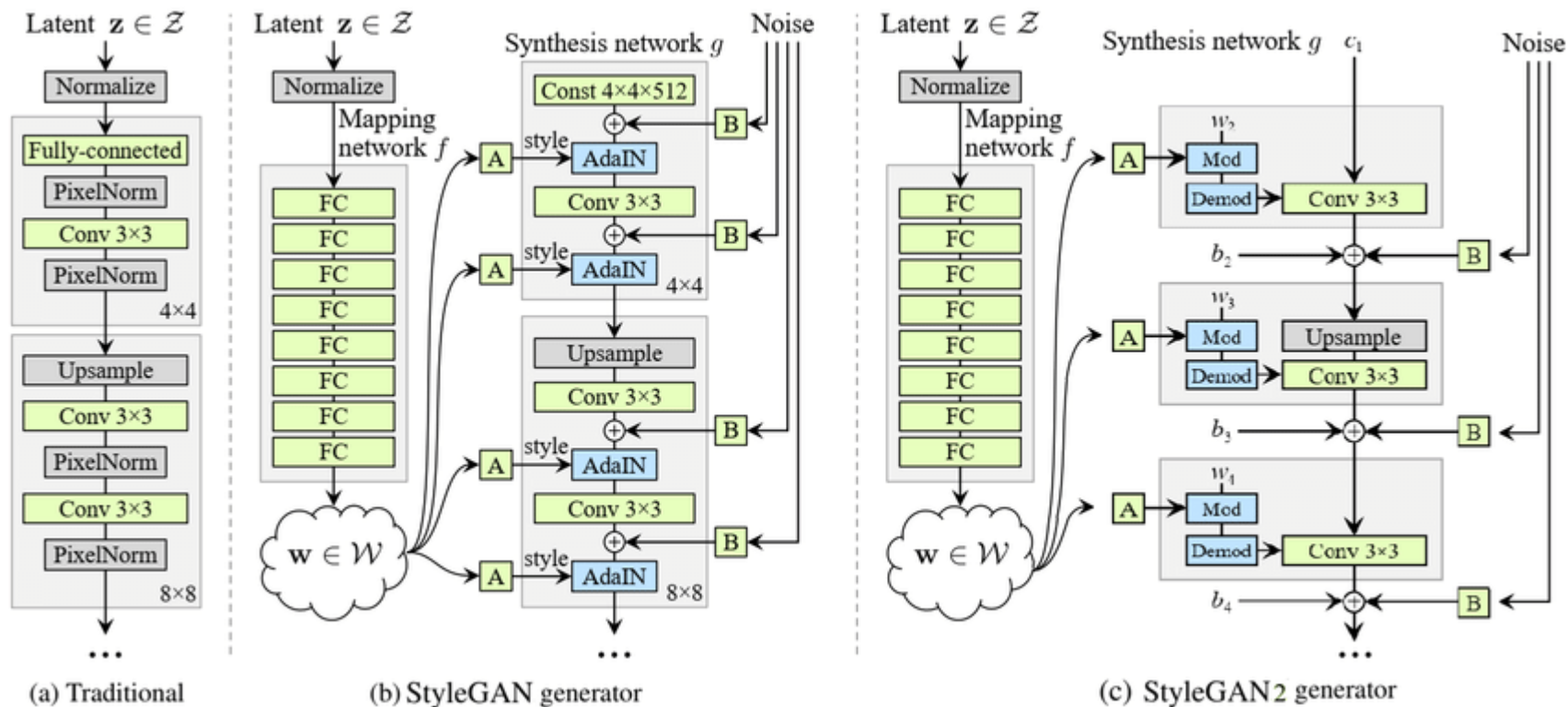
# 02. Prerequisite

## 2. StyleGAN



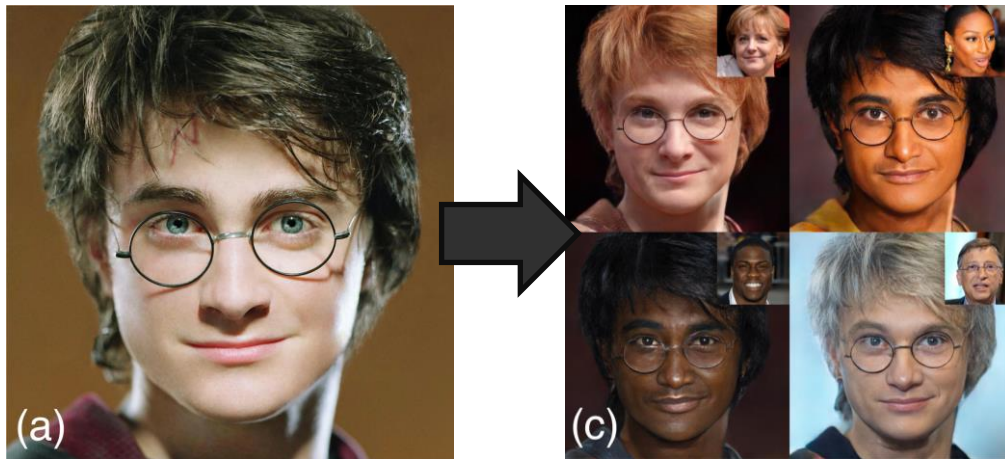
# 02. Prerequisite

## 2. StyleGAN



# 02. Prerequisite

## 2. StyleGAN



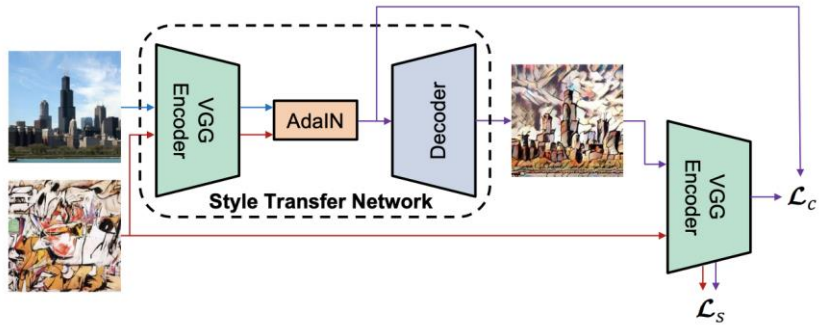
► Disentangle

► Vector Operation

# 02. Prerequisite

## 2. StyleGAN

### ► GAN Inversion



Our configuration F — generated images



Our configuration F — real images



**Q & A**

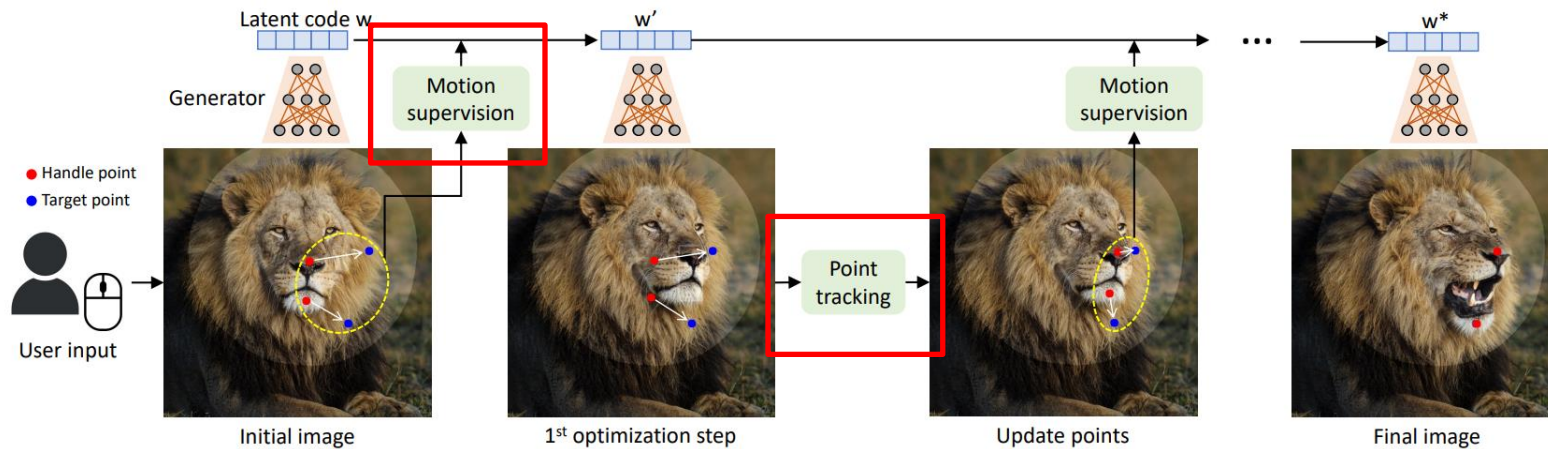
**—**

**03**

# **Methods**

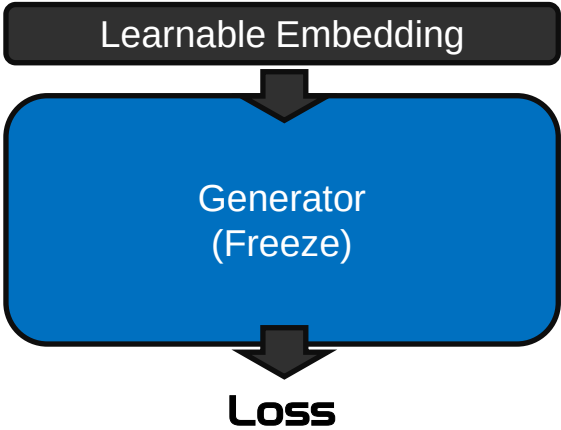
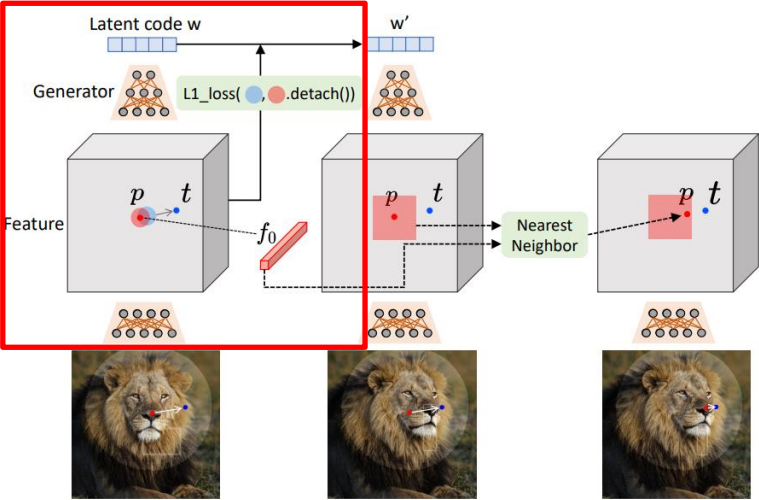
# 03. Methods

## 1. Pipeline



# 03. Methods

## 2. Motion Supervision



단위 벡터만큼 이동한 패치와의 Feature 차이

Point 갯수  $n$

$$\mathcal{L} = \sum_{i=0}^n \sum_{q_i \in \Omega_1(p_i, r_1)} \|F(q_i) - F(q_i + d_i)\|_1 + \lambda \| (F - F_0) \cdot (1 - M) \|_1$$

일정반경 내 Patch

Mask 설정하였을 때 Mask 안에만 수정이 이루어 지도록

# 03. Methods

## 2. Motion Supervision

$$\mathcal{L} = \sum_{i=0}^n \sum_{\mathbf{q}_i \in \Omega_1(\mathbf{p}_i, r_1)} \|\mathbf{F}(\mathbf{q}_i) - \mathbf{F}(\mathbf{q}_i + \mathbf{d}_i)\|_1 + \lambda \|(\mathbf{F} - \mathbf{F}_0) \cdot (1 - \mathbf{M})\|_1$$

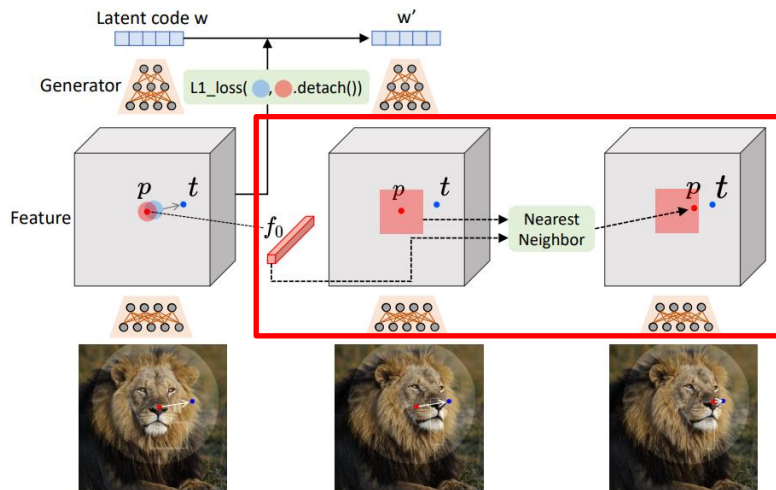
```
# Motion supervision
loss_motion = 0
res.stop = True
for j, point in enumerate(points):
    direction = torch.Tensor([targets[j][1] - point[1], targets[j][0] - point[0]])
    if torch.linalg.norm(direction) > max(2 / 512 * h, 2):
        res.stop = False
    if torch.linalg.norm(direction) > 1:
        distance = ((xx.to(self._device) - point[0])**2 + (yy.to(self._device) - point[1])**2)**0.5
        relis, reljs = torch.where(distance < round(r1 / 512 * h))
        direction = direction / (torch.linalg.norm(direction) + 1e-7)
        gridh = (relis - direction[1]) / (h - 1) * 2 - 1
        gridw = (reljs - direction[0]) / (w - 1) * 2 - 1
        grid = torch.stack([gridw, gridh], dim=-1).unsqueeze(0).unsqueeze(0)
        target = F.grid_sample(feats_resize.float(), grid, align_corners=True).squeeze(2)
        loss_motion += F.l1_loss(feats_resize[:, :, relis, reljs], target.detach())

loss = loss_motion
if mask is not None:
    if mask.min() == 0 and mask.max() == 1:
        mask_usq = mask.to(self._device).unsqueeze(0).unsqueeze(0)
        loss_fix = F.l1_loss(feats_resize * mask_usq, self.feats_resize * mask_usq)
        loss += lambda_mask * loss_fix

loss += reg * F.l1_loss(ws, self.w0) # latent code regularization
if not res.stop:
    self.w_optim.zero_grad()
    loss.backward()
    self.w_optim.step()
```

# 03. Methods

## 3. Point Tracking



$$p_i := \arg \min_{q_i \in \Omega_2(p_i, r_2)} \|F'(q_i) - f_i\|_1.$$

▶ 가장 Feature가 이전과 비슷한 곳으로 Handle Point를 Update

```
# Point tracking with feature matching
with torch.no_grad():
    for j, point in enumerate(points):
        r = round(r2 / 512 * h)
        up = max(point[0] - r, 0)
        down = min(point[0] + r + 1, h)
        left = max(point[1] - r, 0)
        right = min(point[1] + r + 1, w)
        feat_patch = feat_resize[:, :, up:down, left:right]
        L2 = torch.linalg.norm(feat_patch - self.feats_refs[j].reshape(1, -1, 1, 1), dim=1)
        _, idx = torch.min(L2.view(1, -1), -1)
        width = right - left
        point = [idx.item() // width + up, idx.item() % width + left]
        points[j] = point
```

**04**

# **Experiments**

# 04. Experiments

## 1. Datasets

- 1) FFHQ (Flickr-Faces-HQ Dataset)
- 2) AFHQCat (Animal Faces-HQ Dataset)
- 3) SHHQ (StyleGAN-Human-HQ Dataset)
- 4) LSUN Car / LSUN Cat
- 5) Landscape HQ ...





# 04. Experiments

## 2. Qualitative Results

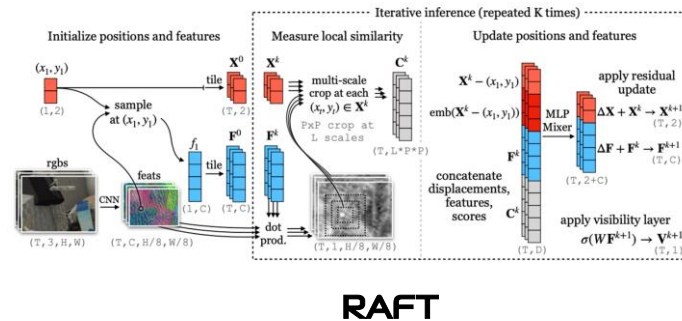
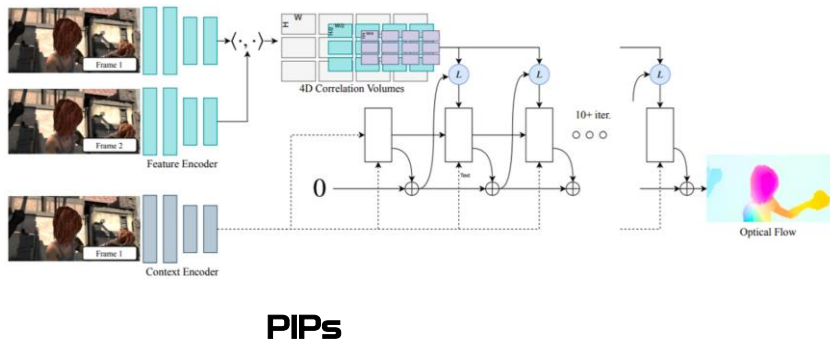
### ► Vs. UserControllableLT



# 04. Experiments

## 2. Qualitative Results

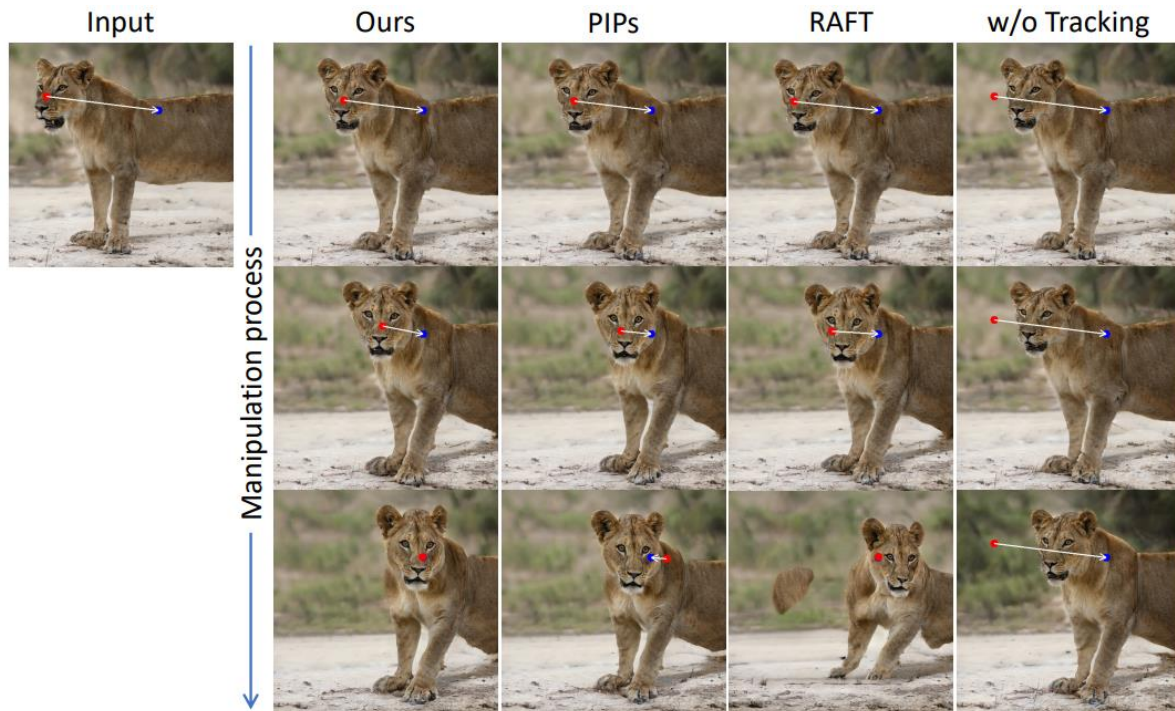
### ► Vs. PIPs / RAFT



# 04. Experiments

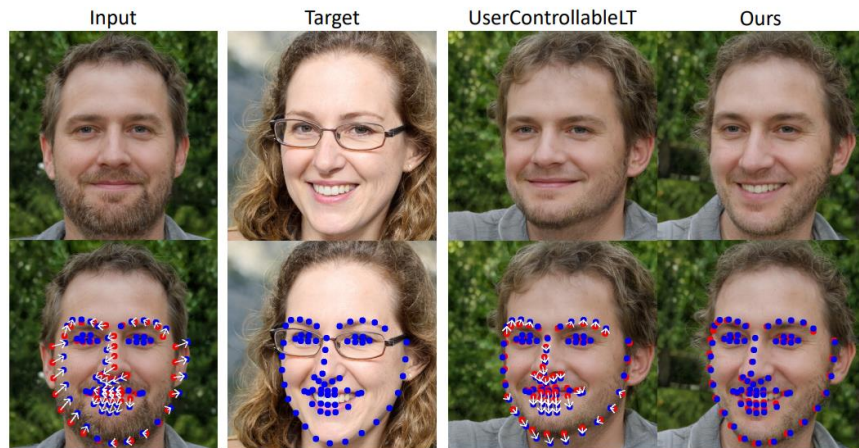
## 2. Qualitative Results

### ► Vs. PIPs / RAFT



# 04. Experiments

## 3. Qualitative Results



evaluation on face keypoint manipulation. We compare between edited points and target points. The FID and Time are reported based on the ‘1 point’ setting.

| Method                | 1 point     | 5 points    | 68 points   | FID         | Time (s) |
|-----------------------|-------------|-------------|-------------|-------------|----------|
| No edit               | 12.93       | 11.66       | 16.02       | -           | -        |
| UserControllableLT    | 11.64       | 10.41       | 10.15       | 25.32       | 0.03     |
| Ours w. RAFT tracking | 13.43       | 13.59       | 15.92       | 51.37       | 15.4     |
| Ours w. PIPs tracking | 2.98        | 4.83        | 5.30        | 31.87       | 6.6      |
| Ours                  | <b>2.44</b> | <b>3.18</b> | <b>4.73</b> | <b>9.28</b> | 2.0      |

# 04. Experiments

## 3. Qualitative Results

Table 2. Quantitative evaluation on paired image reconstruction. We follow the evaluation in [Endo 2022] and report MSE ( $\times 10^2$ ) $\downarrow$  and LPIPS ( $\times 10$ ) $\downarrow$  scores.

| Dataset               | Lion        |             | LSUN Cat    |             | Dog         |             | LSUN Car    |             |
|-----------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| Metric                | MSE         | LPIPS       | MSE         | LPIPS       | MSE         | LPIPS       | MSE         | LPIPS       |
| UserControllableLT    | 1.82        | 1.14        | 1.25        | 0.87        | 1.23        | 0.92        | 1.98        | 0.85        |
| Ours w. RAFT tracking | 1.09        | 0.99        | 1.84        | 1.15        | 0.91        | 0.76        | 2.37        | 0.94        |
| Ours w. PIPs tracking | 0.80        | 0.82        | 1.11        | 0.85        | 0.78        | 0.63        | 1.81        | 0.79        |
| Ours                  | <b>0.66</b> | <b>0.72</b> | <b>1.04</b> | <b>0.82</b> | <b>0.48</b> | <b>0.44</b> | <b>1.67</b> | <b>0.74</b> |

**05**

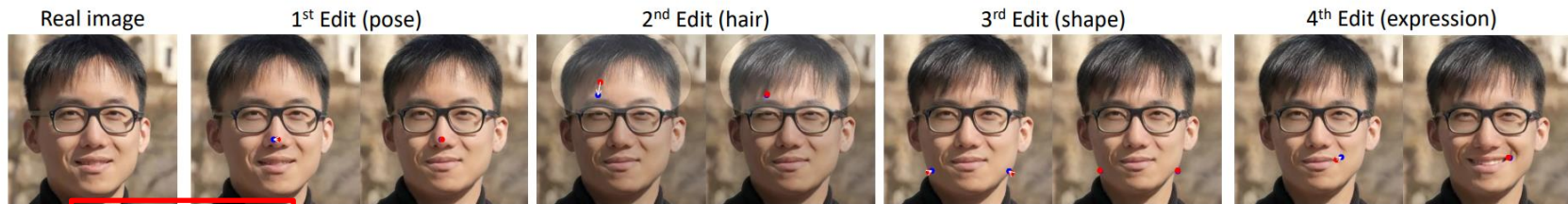
# **Conclusion**



# 05. Conclusion

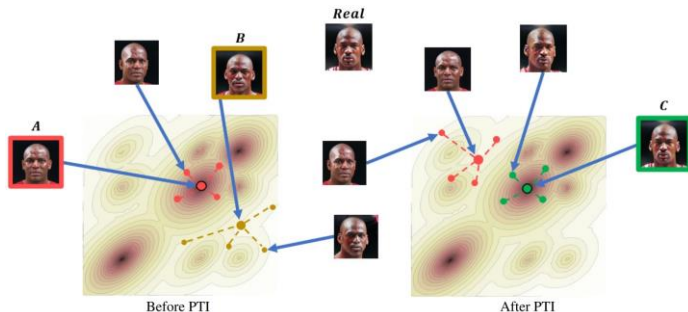
## 1. Potential

- ▶ 공개 당시, 포토샵과 같은 이미지 편집 툴에서 당장이라도 사용될 수 있을 것이라는 기대감과 함께 큰 인기를 끌
- ▶ 하지만, 실제 Application에서 당장 활용하기에는 기술적 한계가 존재



GAN Inversion

PTI Inversion



# 05. Conclusion

## 1. Potential



GAN Inversion

**Inversion된 이미지와 실제 이미지는  
엄밀하게 말해 다른 이미지**



GAN Inversion



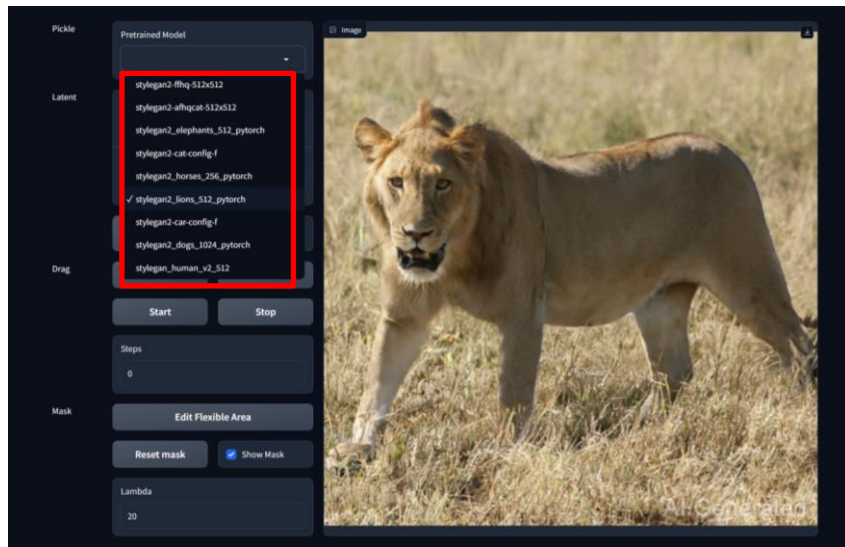
# 05. Conclusion

## 1. Potential

- ▶ Edit가 잘 되는 이미지는 GAN으로 생성한 이미지
- ▶ StyleGAN2에서 또한 현재 Inversion의 한계를 명확하게 언급
- ▶ 본 논문은 Inversion 논문은 아님
- ▶ Latent Vector를 다루는 새로운 방법을 제시한 점은 큰 Contribution이 있음

# 05. Conclusion

## 2. Versatility



▶ Face 영상을 편집하기 위해서는 Face 영상에서 학습된 Pretrain Model을 사용

▶ 대용량의 여러 학습데이터를 학습하여 하나의 모델로 모든 입력을 대응 가능한 GPT와 같이 범용성을 가지기는 어려워 보임

**Q & A**

**—**

**Thank you for your attention**